

Obsah

1	ISSET (PROMĚNNÁ) – ZDA JE PROMĚNNÁ INICIALIZOVANÁ.....	3
2	PŘÍKAZ DIE.....	3
2.1	POUŽITÍ.....	3
1	DATABÁZE MYSQL.....	3
1.1	ZÁKLADNÍ PŘÍKAZY PRO PRÁCI S DATABÁZÍ.....	3
1.2	TABULKY V MYSQL.....	4
1.2.1	Vytvoření tabulky.....	4
1.2.2	Výpis popisu tabulky.....	4
1.2.3	Změny v tabulce.....	5
1.2.4	Nový sloupec.....	5
1.2.5	Zamykání tabulek.....	5
1.2.6	Smazání tabulky.....	5
1.3	DATOVÉ TYPY V TABULCE.....	6
1.3.1	Základní informace.....	6
1.3.2	Celá čísla.....	6
1.3.3	Čísla s pohyblivou desetinou čárkou.....	6
1.3.4	Datum a čas.....	6
1.3.5	Řetězce.....	6
1.3.6	Modifikátory.....	7
1.4	PRÁCE S DATY.....	7
1.4.1	Vkládání záznamů.....	7
1.4.2	Vkládání záznamů ze souboru.....	8
1.4.3	Obnova záznamů.....	8
1.4.4	Výpis záznamů.....	8
1.4.5	Výběr části dat podle podmínky.....	9
1.4.6	Pojmenování.....	9
1.4.7	Mezi hodnotami.....	9
1.4.8	Spojení proměnných.....	9
1.4.9	Odstranění duplikátů.....	10
1.4.10	Slučování do skupin.....	10
1.4.11	V hodnotách.....	10
1.4.12	Omezení počtu.....	10
1.4.13	Seřazení.....	10
1.4.14	Logické operátory.....	11
1.4.15	Kontrolní funkce.....	11
1.4.16	Manipulace s čísly (agregační funkce).....	11
1.4.17	Manipulace s textem.....	12
1.4.18	Manipulace s datumem a časem.....	12
1.4.19	Výpis záznamů do souboru.....	13
1.4.20	Mazání záznamů.....	13
2	APLIKCE PHPMYADMIN.....	13
3	NASTAVENÍ RELACE MEZI TABULKAMI MASQL.....	14

3.1	PŘÍKLAD.....	14
-----	--------------	----

1 Isset (proměnná) – zda je proměnná inicializovaná

empty(proměnná) - zda není proměnná inicializovaná

2 Příkaz die

- Používá se v příkazu if.
- Pokud použijete příkaz die, PHP skript okamžitě "umírá" a vypíše poslední text, který napíšete jako parametr tohoto příkazu.
- Cokoli dalšího, co je za příkazem die už se neprovádí, nedojde na to řada, protože skript už "umřel", sotva narazil na příkaz die:

```
<html>
<head>
<title>Příklad 4. z 5. dílu</title>
<meta http-equiv="Content-Type" content="text/html; charset=windows-1250">
</head>
<body>
<h1><?php
    echo "Ještě žiju!<br>";
    die("Právě umírám! Tohle je moje poslední věta!<br>");
    echo "Tohle už nikdy nevypíšu<br>";
?></h1>
</body>
</html>
```

2.1 Použití

v případech, kdy nastane ve skriptu chyba, a pokračovat dál ve vykonávání PHP skriptu už nemá smysl.

Například můžete chtít vypsat obsah souboru, který už neexistuje. Nebo nastane jiná chyba. Něco se někde ztratí, není k dispozici, prostě cokoliv vám zabráni úspěšně dokončit to, co jste v PHP naprogramovali.

3 Kontrola formuláře

`$_SERVER ["PHP_SELF"]` je super globální proměnná, která vrací název souboru aktuálně spouštěcího skriptu.

Jaká je funkce htmlspecialchars ()?

Funkce htmlspecialchars () převede speciální znaky na entity HTML. To znamená, že nahradí znaky HTML, jako jsou <a>, s < a>. To zabraňuje útočníkům, aby využívali kód vložením kódu HTML nebo kódu Javascript (útoky skriptování mezi počítači) do formulářů.

3.1 Jak se vyhnout \$_SERVER ["PHP_SELF"] Využívá?

Pomocí funkce htmlspecialchars () lze zabránit zneužití \$_SERVER ["PHP_SELF"].

Kód formuláře by měl vypadat takto:

```
<form method="post" action="<?php echo htmlspecialchars  
($_SERVER["PHP_SELF"]);?>">
```

Funkce htmlspecialchars () převede speciální znaky na entity HTML. Pokud se uživatel pokusí využít proměnnou PHP_SELF, bude to mít za následek následující výstup:

```
<form method="post" action="test_form.php/  
&quot;&gt;&lt;script&gt;alert('hacked')&lt;/script&gt;">
```

4 Kontrola formuláře na neprázdnost

<https://195.113.207.163/~smrcka/wt1/kontrola.php>

```
<!DOCTYPE HTML>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
.error {color: #FF0000;}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<?php
```

```
// define variables and set to empty values
```

```
$nameErr = $emailErr = $genderErr = $websiteErr = "";
```

```
$name = $email = $gender = $comment = $website = "";
```

```
if ($_SERVER["REQUEST_METHOD"] == "POST") {
```

```
    if (empty($_POST["name"])) {
```

```
        $nameErr = "Name is required";
```

```
    } else {
```

```
        $name = test_input($_POST["name"]);
```

```
    }
```

```
if (empty($_POST["email"])) {
```

```
    $emailErr = "Email is required";
```

```
} else {
```

```
    $email = test_input($_POST["email"]);
```

```
}
```

```
if (empty($_POST["website"])) {
```

```
    $website = "";
```

```
} else {
```

```
    $website = test_input($_POST["website"]);
```

```
}
```

```
if (empty($_POST["comment"])) {
```

```
    $comment = "";
```

```
} else {
```

```
    $comment = test_input($_POST["comment"]);
```

```
}
```

```
if (empty($_POST["gender"])) {
```

```
    $genderErr = "Gender is required";
```

```
} else {
```

```
    $gender = test_input($_POST["gender"]);
```

```
}
```

```
}
```

```
function test_input($data) {
    $data = trim($data);
    $data = stripslashes($data);
    $data = htmlspecialchars($data);
    return $data;
}
?>
```

<h2>PHP Form Validation Example</h2>

<p>* required field</p>

<form method="post" action="<?php echo htmlspecialchars(\$_SERVER["PHP_SELF"]);?>">

Name: <input type="text" name="name">

* <?php echo \$nameErr;?>

E-mail: <input type="text" name="email">

* <?php echo \$emailErr;?>

Website: <input type="text" name="website">

<?php echo \$websiteErr;?>

Comment: <textarea name="comment" rows="5" cols="40"></textarea>

Gender:

<input type="radio" name="gender" value="female">Female

<input type="radio" name="gender" value="male">Male

<input type="radio" name="gender" value="other">Other

* <?php echo \$genderErr;?>

<input type="submit" name="submit" value="Submit">

</form>

<?php

echo "<h2>Your Input:</h2>";

```

echo $name;
echo "<br>";
echo $email;
echo "<br>";
echo $website;
echo "<br>";
echo $comment;
echo "<br>";
echo $gender;
?>

```

```
</body>
```

```
</html>
```

<https://195.113.207.163/~smrcka/wt1/kontrola.php>

5 Kontrola formuláře – regulární výraz

6 Regulární výrazy

Používají se pro kontrolu nebo hledání obsahu řetězců. V následující tabulce jsou uvedeny třídy znaků použitelných v regulárních výrazech.

Výraz	Znamená	Vyhovují	Nevyhovují
abc	Řetězec, který obsahuje tento podřetězec	abc, aascvabcmn,abcd	abbbvvvc, cba,
^ ^abc	Začátek řetězce	abcd,abc,	bbabc, nnabc
\.	tečka		

\\	lomítko		
*	výraz se může opakovat, nebo nemusí být		
a*b		aaaabbbb,ab,agb	aaaadd, a
+	výraz se musí objevit alespoň jednou		
.	libovolný znak		
?	výraz není, nebo je jedenkrát		
\$	konec řetězce	hasghaa,a	aafdgd, cc, an
a\$			
a{2,6}	minimální a maximální počet opakování a	aa,aaa,aaaa,aaaaa,aaaaaa	a,aaaaaaa
a{5,}	a je minimálně 5 krát		
a{5}	a je maximálně 5 krát		
[abcd]	jeden znak z množiny		
[a-d]	zkrácený výpis předchozího řádku		
[a-z]	malé písmena		
[A-Z]	velké písmeno		
[a-zA-Z]	malé nebo velké písmeno		
[^ab]	regulární výraz vyhoví libovolnému znaku, kromě těch v závorce		
	rozdělení reg. výrazu na podvýrazy	Po, Ut, St, Ct, Pa, So, Ne	Pon, So, Adhg,12
Po Ut St Ct Pa So Ne	řetězec vyhovuje, pokud vyhovuje alespoň jedné části		
(Po Ut St Ct Pa So Ne) (Mo Th We Th Fr Sa Su)		Po,St, Mo,.....	
[[:<:]]Novak[[:>:]]	vyhoví pouze řetězec Novak	Novak	Novakova, Novacek,

alnum	Alfanumerické znaky (písmena a číslice)
alpha	písmena anglické abecedy
blank	mezera a tabulátor
cntrl	řídící znaky
digit	Číslice
graph	všechny znaky, které mají grafické znázornění
lower	malá písmena anglické abecedy
print	tisknutelné znaky
punct	interpunkční a další znaky (závorky, zavináče,...)
space	mezera, tab, nová řádka, nová stránka,...
upper	Velká písmena anglické abecedy
xdigit	Šestnáctkové číslice (0-9, a-f, A-F)

[[[:<:]]][[:digit:]] {4} [[[:>:]]]	Řetězec obsahující 4 místné číslo
---------------------------------------	-----------------------------------

7 Použití regulárního výrazu

Starší verze

```
<?
    $neco = "cde";
    if (EReg("[ab]", $neco, $Cast))
        echo "reg. vyraz je spravny";
    else echo "reg vyraz spatny";
?>
```

Novější verze

```
$name="Franta"
if (!preg_match("/^[a-zA-Z ]*$/", $name)) {
    $nameErr = "Only letters and white space allowed";
}
```

PHP - Uchovávejte hodnoty ve formuláři a kontrola na reg. Výraz

```
<!DOCTYPE HTML>
<html>
<head>
<style>
.error {color: #FF0000;}
</style>
</head>
<body>

<?php
// define variables and set to empty values
$nameErr = $emailErr = $genderErr = $websiteErr = "";
$name = $email = $gender = $comment = $website = "";

if ($_SERVER["REQUEST_METHOD"] == "POST") {
    if (empty($_POST["name"])) {
        $nameErr = "Name is required";
    } else {
        $name = test_input($_POST["name"]);
        // check if name only contains letters and whitespace
        if (!preg_match("/^[a-zA-Z ]*$/", $name)) {
            $nameErr = "Only letters and white space allowed";
        }
    }
}

if (empty($_POST["email"])) {
```

```

$emailErr = "Email is required";
} else {
    $email = test_input($_POST["email"]);
    // check if e-mail address is well-formed
    if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
        $emailErr = "Invalid email format";
    }
}

if (empty($_POST["website"])) {
    $website = "";
} else {
    $website = test_input($_POST["website"]);
    // check if URL address syntax is valid (this regular expression also allows dashes in
the URL)
    if (!preg_match("/\b(?:https? | ftp):\/\/ | www\.)[-a-z0-9+&@#\/%?=_|!:,;]*[-a-z0-
9+&@#\/%?=_|!]/i",$website)) {
        $websiteErr = "Invalid URL";
    }
}

if (empty($_POST["comment"])) {
    $comment = "";
} else {
    $comment = test_input($_POST["comment"]);
}

if (empty($_POST["gender"])) {
    $genderErr = "Gender is required";
} else {
    $gender = test_input($_POST["gender"]);
}
}

function test_input($data) {
    $data = trim($data);
    $data = stripslashes($data);
    $data = htmlspecialchars($data);
    return $data;
}
?>

```

<h2>PHP Form Validation Example</h2>

<p>* required field</p>

<form method="post" action="<?php echo htmlspecialchars(\$_SERVER["PHP_SELF"]);?>

```

">
Name: <input type="text" name="name" value="<?php echo $name;?>">
<span class="error">* <?php echo $nameErr;?></span>
<br><br>
E-mail: <input type="text" name="email" value="<?php echo $email;?>">
<span class="error">* <?php echo $emailErr;?></span>
<br><br>
Website: <input type="text" name="website" value="<?php echo $website;?>">
<span class="error"><?php echo $websiteErr;?></span>
<br><br>
Comment: <textarea name="comment" rows="5" cols="40"><?php echo $comment;?
</textarea>
<br><br>
Gender:
<input type="radio" name="gender" <?php if (isset($gender) &&
$gender=="female") echo "checked";?>value="female">Female
<input type="radio" name="gender" <?php if (isset($gender) &&
$gender=="male") echo "checked";?>value="male">Male
<input type="radio" name="gender" <?php if (isset($gender) &&
$gender=="other") echo "checked";?>value="other">Other
<span class="error">* <?php echo $genderErr;?></span>
<br><br>
<input type="submit" name="submit" value="Submit">
</form>

<?php
echo "<h2>Your Input:</h2>";
echo $name;
echo "<br>";
echo $email;
echo "<br>";
echo $website;
echo "<br>";
echo $comment;
echo "<br>";
echo $gender;
?>

</body>
</html>

```

8 DATABÁZE MySQL

8.1 Základní příkazy pro práci s databází

Připojení k MySQL (MariaDB)

Příkazový řádek: `mysql -u uživatel -p`

připojení ke konkrétní databázi

`use jméno_databáze`

9 Aplikce PHPMyadmin

Aplikace umožňuje správu MySQL databáze.

<https://alpha.kts.vspj.cz/VSPJphpMyAdmin/>

Další aplikací pro správu je adminer

Výpis databází MySQL

`SHOW DATABASES` - příkaz zobrazí jména databází ve vašem spuštěném MySQL

Založení databáze

`CREATE DATABASE nazev_databaze;root`

- Příkaz vytvoří databázi se jménem "nazev_databaze".
- Abychom mohli databázi používat musíme v ní vytvořit jednu či více tabulek příkazem `CREATE TABLE`.
- Při práci v příkazovém řádku musíme určit aktivní databázi příkazem `USE`.

USE nazev_databaze;

Databázi "nazev_databaze" nastavíme takto jako aktivní a můžeme s ní pracovat.

Název aktuální databáze

`SELECT DATABASE();`

Výpis seznamu tabulek v databázi

`SHOW TABLES;`

`SHOW TABLES FROM nazev_databaze;`

Smazání databáze

`DROP DATABASE nazev_databaze;`

9.1 TABULKY v MySQL

9.1.1 Vytvoření tabulky

`CREATE TABLE nazev_tabulky (nazev_sloupce datovy_typ,... );`

Příklad

```
CREATE TABLE knihovna (autor VARCHAR(20), kniha VARCHAR(20) NOT NULL  
PRIMARY KEY, stran SMALLINT UNSIGNED, rok YEAR(4), poznamka  
ENUM('neprecteno', 'precteno', 'pujmeno') DEFAULT 'neprecteno');
```

Poznámka – datové typy polí jsou vysvětleny v následující kapitole.

Vytvoření dočasné tabulky

```
CREATE TEMPORARY TABLE nazev_tabulky (nazev_sloupce datovy_typ,... );
```

Dočasná tabulka po uzavření spojení s databází zanikne.

9.1.2 Výpis popisu tabulky

```
DESCRIBE nazev_tabulky;
```

```
SHOW COLUMNS FROM nazev_tabulky;
```

Příkaz m zobrazí definici požadované tabulky (názvy + datové typy + modifikátory).

9.1.3 Změny v tabulce

```
ALTER TABLE nazev_tabulky prikaz1, prikaz2, prikaz3, ..;
```

Provede nějaký příkaz "prikaz" s tabulkou "nazev_tabulky", příkazy jsou následně popsány:

9.1.4 Nový sloupec

```
ADD nazev_noveho_sloupce datovy_typ;
```

```
ADD COLUMN nazev_noveho_sloupce datovy_typ; - příkaz přidá do tabulky nový sloupec
```

Příklad

```
ALTER TABLE knihovna ADD COLUMN vydavatel VARCHAR(10);
```

Modifikátory

FIRST- přidá nový sloupec na začátek tabulky

Příklad

```
ALTER TABLE knihovna ADD COLUMN cislo SMALLINT FIRST;
```

```
AFTER nazev_sloupce - přidá nový sloupec za sloupec "nazev_sloupce"
```

Příklad

```
ALTER TABLE knihovna ADD COLUMN zavr VARCHAR(10) AFTER kniha;
```

Smazání sloupce

```
DROP nazev_odstranovaneho_sloupce;
```

```
DROP COLUMN nazev_odstranovaneho_sloupce - příkaz odebere požadovaný sloupec
```

Příklad: **ALTER TABLE** knihovna **DROP** vydavatel;

Změna parametrů

```
CHANGE nazev_sloupce novy_nazev_sloupce nove_nastaveni - změní datový typ a může  
sloupec i přejmenovat.
```

Příklad: **ALTER TABLE** knihovna **CHANGE** kniha knihy VARCHAR(30) NOT NULL;

Modifikace parametrů

```
MODIFY nazev_sloupce nove_nastaveni - u požadovaného sloupce změní datový typ
```

Příklad: **ALTER TABLE** knihovna **MODIFY** kniha VARCHAR(30) NOT NULL;

Přejmenování tabulky

RENAME *novy_nazev_tabulky*;

Příklad: ALTER TABLE knihovna RENAME knihovnicka;

9.1.5 Zamykání tabulek

LOCK TABLES *nazev_tabulky* **READ**, *nazev_tabulky* **WRITE** - uzamkne vyjmenované tabulky pro čtení (READ), nebo zápis (WRITE). Po uzamknutí mají právo čtení, nebo zápisu v tabulce pouze ty příkazy, které se nachází mezi LOCK ... UNLOCK.

UNLOCK TABLES - odemčení všech zamčených tabulek

9.1.6 Smazání tabulky

DROP TABLE *nazev_tabulky* - odstraní z aktivní databáze tabulku s názvem "*nazev_tabulky*"

9.2 DATOVÉ TYPY v tabulce

9.2.1 Základní informace

Pro omezení délky řetězce (maximální velikost je 255) používáme parametr "*m*", zápis je: *datovy_typ*(*m*).

Příklad: TINYINT(1), nebo VARCHAR(100)

U reálných čísel používáme navíc parametr "*d*" (maximální velikost je 30). Tímto parametrem omezíme délku čísla za desetinou čárkou, zápis je: *datovy_typ*(*m*,*d*)

Příklad: FLOAT(5,3)

9.2.2 Celá čísla

TINYINT- rozsah hodnot od -128 do +127, bez znaménka 0 až 255

SMALLINT- rozsah hodnot od -32768 do 32767, bez znaménka 0 až 65535

MEDIUMINT- rozsah hodnot od -8388608 do +8388607, bez znaménka 0 až 16777215

INT nebo **INTEGER** - rozsah hodnot od -2147483648 do +2147483647, bez znaménka 0 až 4294967295

BIGINT - rozsah hodnot od -9223372036854775808 do +9223372036854775807, bez znaménka (UNSIGNED) tedy 0 až 18446744073709551615

BIT nebo **BOOL** - synonymum pro TINYINT(1)

9.2.3 Čísla s pohyblivou desetinou čárkou

FLOAT- rozsah hodnot od -3.402823466E+38 do 3.402823466E+38

DOUBLE - rozsah hodnot od -1.7976931348623157E+308 do 1.7976931348623157E+308

DOUBLE, PRECISION nebo **REAL** - synonyma pro typ DOUBLE

DECIMAL(m,d) - rozsah nastavíme parametry "*m*" a "*d*", maximální rozsah je stejný s typem DOUBLE

DEC(m,d) nebo **NUMERIC(m,d)** - synonyma pro typ DECIMAL(m,d)

9.2.4 Datum a čas

DATE - datum ve formátu "rok-měsíc-den" respektive "RRRR-MM-DD" a v rozsahu 1000- 01-1 až 9999-12-31

DATETIME - datum a čas v rozsahu 1000-01-01 00:00:00 až 9999-12-31 23:59:59 (formát je "RRRR-MM-DD HH:MM:SS")

TIMESTAMP(m) - datum a čas v rozsahu 1970-01-01 00:00:00 až 2037-01-01 00:00:00
Pokud uvedeme parametr "m" s přípustnými hodnotami 14 (nebo chybějící), 12, 8, či 6 nastavíme si formát. Formát pak bude "RRRRMMDDHHMMSS", "RRMMDDHHMMSS", "RRRRMMDD", či "RRMMDD". Pokud do buňky tohoto typu nic nezapíšeme, MySQL sám doplní aktuální čas změny v daném řádku.

TIME - časový rozsah je -838:59:59 až 838:59:59 a formát datového typu "HH:MM:SS"

YEAR(m) - při YEAR(4) bude rozsah 1901 až 2155, při YEAR(2) bude rozsah 1970-2069, formát je "RRRR"

9.2.5 Řetězce

CHAR(m) - délka řetězce "m" může být v rozsahu 0-255

Pokud je vložený řetězec kratší než nastavíme, chybějící znaky jsou automaticky doplněny mezerami. CHAR (tedy bez "m") je považováno za CHAR(1).

VARCHAR(m) - délka řetězce "m" může být v rozsahu 0-255

TINYBLOB, TINYTEXT - délka řetězce je maximálně 255 znaků

BLOB, TEXT - délka řetězce je maximálně 65535 znaků

MEDIUMBLOB, MEDIUMTEXT - délka řetězce je maximálně 16777215 znaků

LOB, LONGTEXT - délka řetězce je maximálně 4294967295 znaků

ENUM('prvek1','prvek2',...) - pole předem definovaných řetězců (prvků) o maximálním počtu 65535

- V buňce tabulky pak může být pouze jeden z prvků, které jsem předdefinovali.
- Místo názvů prvků můžeme používat i jejich pořadí, tedy: 1 (místo 'prvek1'), 2 (místo 'prvek2')...

SET('prvek1','prvek2',...) - pole předem definovaných řetězců (prvků) o maximálním počtu 64. V buňce tabulky pak může být i více z prvků, které jsem předdefinovali.

9.2.6 Modifikátory

AUTO_INCREMENT - systém si sám ve sloupci generuje unikátní (jedinečné) číselné hodnoty. Modifikátor lze použít pouze na celočíselný datový typ.

BINARY - pro CHAR a VARCHAR; tento typ bude brán jako binární a budou se tak rozlišovat malá a velká písmena.

DEFAULT vychodi_hodnota - pokud bude buňka prázdná, systém do ní automaticky přiřadí hodnotu "vychodi_hodnota".

NOT NULL - pokud použijeme tento modifikátor, označený typ bude muset v každé buňce obsahovat nějakou hodnotu.

NULL - opak NOT NULL, buňka může být prázdná.

PRIMARY KEY - označený typ bude sloužit jako primární klíč, při jeho použití musíme zároveň použít UNIQUE. Sloupec s PRIMAR KEY nám tedy jedinečným způsobem identifikuje záznamy v tabulce.

UNIQUE - v daném sloupci nesmějí být v buňkách stejné hodnoty, tedy co kus to unikát.

UNSIGNED - pokud použijeme modifikátor UNSIGNED, datový typ bude bez znaménka a posune se interval hodnot.

U čísel s pohyblivou desetinou čárkou se interval použitím UNSIGNED neposunuje a bereou se jen kladná čísla. TINYINT má rozsah -118 až +127 a TINYINT UNSIGNED má rozsah 0 až 255.

ZEROFILL - použití u čísel, příkaz doplní před číslo nuly v celé jeho šířce.

Příklad: pokud máme definováno MEDIUMINT(6) ZEROFILL a je v něm hodnota 123, tak se nám zobrazí 000123.

9.3 PRÁCE S DATY

9.3.1 Vkládání záznamů

INSERT INTO *nazev_tabulky* **VALUES** (*seznam_hodnot*);

Pro všechny sloupce v tabulce "*nazev_tabulky*" musíme vložit data.

Příklad: INSERT INTO knihovna VALUES ('Oranžový Oto','Tropické ovoce',110,2003,'neprecteno');

Vložení jen do některých sloupců:

INSERT INTO knihovna (autor,kniha) VALUES ('Oranžový Oto','Tropické ovoce');

9.3.2 Vkládání záznamů ze souboru

LOAD DATA LOCAL INFILE '*jmeno_souboru*' **INTO TABLE** *nazev_tabulky* - příkaz vloží do tabulky "*nazev_tabulky*" data ze souboru "*jmeno_souboru*", který je lokálně uložen na PC.

Příklad: LOAD DATA LOCAL INFILE 'nove_knihy.txt' INTO TABLE knihovna FIELDS TERMINATED BY ',' ENCLOSED BY '"' LINES TERMINATED BY '\n';

- Záznamy jsou v uvozovkách, oddělené čárkou a konce řádků máme zakončené odentrováním.
- Pokud je pořadí sloupců v souboru odlišné, musíme je připsat do závorky za název tabulky

Modifikátory:

FIELDS TERMINATED BY '*neco*' - znak oddělující jednotlivé záznamy, většinou čárka ',' nebo tabulátor '\t'

ENCLOSED BY '*neco*' - znak uzavírající hodnoty záznamů, většinou uvozovky ''

LINES TERMINATED BY '*neco*' - znak ukončující řádky, většinou odentrování '\n'

LOW_PRIORITY

Příklad: LOAD DATA LOW_PRIORITY LOCAL INFILE...

MySQL uloží data do tabulky až s ní přestanou všichni pracovat.

9.3.3 Obnova záznamů

UPDATE *nazev_tabulky* SET *jmeno_sloupce*=*nova_hodnota* WHERE *podminka*;

Příklad: UPDATE knihovna SET stran='260' WHERE kniha='Lesnictví';

U knihy "Lesnictví" jsme upravili počet stran.

9.3.4 Výpis záznamů

SELECT seznam_pozadavku FROM upresnujici_podminky;

Příklad: SELECT (knihovna.autor) as autorkn, (knihy.autor) as autor2 FROM knihovna, knihy;

Tento příkaz zobrazí z tabulky "knihovna" všechny autory

Příklad: SELECT autor,kniha FROM knihovna;

Tento příkaz zobrazí z tabulky "knihovna" všechny autory a knihy.

Seznam příkazů a podmínek následuje:

*** Vyber vše**

Příklad: SELECT * FROM knihovna;

Hvězdička vypíše z tabulky "knihovna" všechna data.

9.3.5 Výběr části dat podle podmínky

WHERE podmínka;

operátory: = (rovno), <> (nerovno), < (menší), <= (menší nebo rovno), > (větší), >= (větší nebo rovno)

SELECT * FROM knihovna WHERE poznamka='precteno' - vypíše všechny informace o knihách které jsou přečtené "precteno".

SELECT kniha FROM knihovna WHERE poznamka='precteno' - vypíše názvy knih, které jsou označeny jako přečtené "precteno".

SELECT knihovna.kniha FROM knihovna,cetba WHERE knihovna.kniha=cetba.kniha - vypíše se názvy knih z knihovny ("knihovna"), které máme v knihovně ("knihovna") a četli jsme je ("cetba").

Výběr dat funkcí JOIN

tabulka1 LEFT JOIN tabulka2 podmínka;

tabulka2 RIGHT JOIN tabulka1 podmínka;

Příklady

SELECT * FROM tabulka1,tabulka2 WHERE tabulka1.id=tabulka2.id;

SELECT * FROM tabulka1 LEFT JOIN tabulka2 ON tabulka1.id=tabulka2.id;

Oba předchozí příklady vykonávají skoro to samé, ale funkce JOIN vrátí odpovídající řádky levé tabulky (tabulka1) bez ohledu na to, zda k těmto řádkům existuje nějaký odpovídající řádek v druhé tabulce (tabulka2). Funkce WHERE totiž nevrátí nulové hodnoty (NULL).

SELECT * FROM tabulka1 LEFT JOIN tabulka2 USING (id) - zkrácený zápis předchozího příkladu.

9.3.6 Pojmenování

jmeno AS nove_jmeno

Příklad: SELECT k.kniha FROM knihovna AS k, cetba AS c WHERE k.kniha=c.kniha;
Pomocí klíčového slova AS můžeme pojmenováním zkrátit zápis předchozího příkladu.

Příklad: SELECT autor,kniha,(cena*0.95) AS 'cena_bez_dph' FROM knihovna;

Příklad: SELECT 'Jmeno:' AS 'jmeno_autor', autor FROM knihovna;
Zde nám MySQL vypíše vedle jmen autorů sloupec s názvem "jmeno_autor" s DEFAULT hodnotou "Jmeno:".

9.3.7 Mezi hodnotami

nazev_sloupce BETWEEN prvni_hodnota AND druha_hodnota;

Příklad: SELECT * FROM knihovna WHERE rok BETWEEN 1990 AND 2000;

Vypis informace o knihách z knihovny, které vyšly mezi roky 1990 (včetně) a 2000 (včetně)

nazev_sloupce NOT BETWEEN prvni_hodnota AND druha_hodnota - opak předešlého

9.3.8 Spojení proměnných

CONCAT(promenne_pro_spojeni)

Příklad: SELECT CONCAT(kniha, ' - ', autor) AS knihautor FROM knihovna;

Vypíše nový sloupec "knihautor", který bude obsahovat data ve formátu: název knihy - název autora

CONCAT_WS(slucovac,promenne)

Příklad: SELECT CONCAT_WS('.', 'www', 'gene', 'cz'); - vypíše: www.gene.cz

9.3.9 Odstranění duplikátů

DISTINCT

Příklad: SELECT DISTINCT poznamka FROM knihovna;

Tento příklad nám vypíše jaké používáme poznámky, tedy P,N,U (bez DISTINCT by vypsál vše: P,N,U,P,P,N).

9.3.10 Slučování do skupin

GROUP BY

Příklad: SELECT poznamka, SUM(stran) AS 'celkem_stran' FROM knihovna GROUP BY poznamka;

Sečte (příkaz SUM) počet stran u knih seskupených dle poznámek (P-přečteno, N-nepřečteno...)

Nulová hodnota

IS NULL

Příklad: SELECT kniha FROM knihovna WHERE stran IS NULL;

Takto zjistíme knihy s nevyplněným políčkem počet stran.

9.3.11 V hodnotách

IN(kde_hledat)

Příklad: SELECT kniha FROM knihovna WHERE rok IN(2001,2002,2003);

MySQL vypíše knihy z let 2001-2003.

NOT IN - opak IN

9.3.12 Omezení počtu

LIMIT start,pocet;

Příklad: SELECT kniha FROM knihovna WHERE poznamka='neprecteno' LIMIT 0,5;

Najde názvy prvních 5 knih, které jsou v poznámce označeny jako nepřečtené.

Upřesnění výběru

LIKE

Příklad: SELECT kniha FROM knihovna WHERE autor LIKE 'Z%';

Operátor LIKE vybere knihy jejichž autor začíná od Z. Procento "%" nahrazuje libovolný počet znaků, podtržítko "_" pouze jeden znak.

9.3.13 Seřazení

ORDER BY podmínka;

Příklad: SELECT * FROM knihovna ORDER BY autor,kniha;

Zde se vybere z tabulky všechny informace a srovná je vzestupně podle jmen autorů a názvů knih.

ORDER BY podmínka DESC;

Příklad:: SELECT * FROM knihovna ORDER BY autor DESC;

Srovná výpis podle autorů, tentokrát sestupně.

ORDER BY RAND(), ORDER BY RAND(N);

Příklad: SELECT kniha FROM knihovna WHERE poznamka='neprecteno' ORDER BY RAND() LIMIT 1;

Výstupem je jedna nepřečtená kniha náhodně vybraná. Zadáním parametru "N" určíme výchozí hodnotu pro výpočet náhodného čísla.

9.3.14 Logické operátory

AND

Příklad: SELECT kniha FROM knihovna WHERE poznamka='neprecteno' AND rok<2000;
AND vybere nepřečtené knihy vydané před rokem 2000.

OR

Příklad: SELECT kniha FROM knihovna WHERE poznamka='neprecteno' OR poznamka='precteno';
Příkaz vybere všechny nepřečtené a přečtené knihy.

9.3.15 Kontrolní funkce

CASE ... END

Příklad: SELECT CASE hledana_hodnota WHEN 1 THEN 'jedna' WHEN 2 THEN 'dva' ELSE 'tri a vice' END;

Pokud hledaná hodnota bude 1 vypíše MySQL "jedna", pokud 2 vypíše "dva", v ostatních případech "tri a vice".

IF(podminka,pravda,nepravda);

Příklad: SELECT IF(10>9,'ano','ne'); - vypíše se "ano"

IFNULL(podminka,vystup_pri_chybe);

Příklad: SELECT IFNULL(1/0,'chyba');

Dělíme nulou - vypíše "chyba"

NULLIF(promenna1,promenna2);

Vrací se promenna1, pokud se promenna1 nerovná promenna2 (v opačném případě vrátí NULL).

9.3.16 Manipulace s čísly (agregační funkce)

AVG(nazev_sloupce) - spočítá průměr numerických hodnot ve sloupci

Příklad: SELECT AVG(stran) FROM knihovna;

COUNT(nazev_sloupce) - spočítá počet hodnot ve sloupci

COUNT(DISTINCT nazev_sloupce) - spočítá počet jedinečných hodnot ve sloupci

GREATEST(hodnota1,hodnota2,hodno...)

Příklad: SELECT GREATEST(10,3,7,24) - vrátí největší hodnotu (24)

Funkce funguje i pro text (při zadání "J","U","N" vrátí U).

LEAST(hodnota1,hodnota2,hodno...)

Příklad: SELECT LEAST(10,3,7,24); - vrátí nejmenší hodnotu (3)

MAX(nazev_sloupce)

Příklad: SELECT kniha, MAX(stran) FROM knihovna - příkaz nám najde knihu s nejvyšším počtem stran.

MIN(nazev_sloupce) - opak MAX(nazev_sloupce)

MOD(delenec,delitel) - zbytek po celočíselném dělení

ROUND(cislo) - zaokrouhlí zadané "cislo" na celé číslo

ROUND(cislo,pocet_mist) - zaokrouhlí "cislo" na zadaný počet desetinných

STD(nazev_sloupce) - spočítá směrodatnou odchylku číselných hodnot ve sloupci

SUM(nazev_sloupce) - provede součet číselných hodnot ve sloupci

9.3.17 Manipulace s textem

LENGTH(retezec) - funkce vrací délku řetězce; v tomto případě je to 7.

Příklad: SELECT LENGTH('abeceda');

LOCATE(co_hledat,v_cem,kde_zacit) - hledá řetězec "ce" v řetězci "abeceda" od pozice 1

Příklad: SELECT LOCATE('ce','abeceda',1); - výsledkem je 4.

SUBSTRING(retezec,kde_zacit)

Příklad: SELECT SUBSTRING('abeceda',4);

Vypíše řetězec od zadané pozice, tedy "ceda".

REPLACE(retezec,co_nahradit,cim_nahradit);

Příklad: SELECT REPLACE('abeceda','abec','nezb');

Nahrazují se části řetězce, vypíše "nezbada".

REVERSE(retezec)

Příklad: SELECT REVERSE('abeceda');

Obrátí řetězec; vypíše "adeceba".

TRIM(retezec);

Příklad: SELECT TRIM(' abeceda ');

Vymaže z řetězce mezery a vypíše "abeceda"

TRIM(BOTH retezec1 FROM retezec2);

Příklad: SELECT TRIM(BOTH 'a' FROM 'abeceda');

vypíše "beced"

TRIM(LEADING retezec1 FROM retezec2);

Příklad: SELECT TRIM(LEADING 'a' FROM 'abeceda');

vypíše "beceda"

TRIM(TRAILING retezec1 FROM retezec2);

Příklad: SELECT TRIM(TRAILING 'a' FROM 'abeceda');

vypíše "abeced"

LTRIM(retezec);

Příklad: SELECT LTRIM(' abeceda ');
vypíše "abeceda"

RTRIM(retezec);

Příklad: SELECT RTRIM(' abeceda ');
vypíše "abeceda"

UPPER(retezec),. LOWER(retezec);

Příklad: SELECT UPPER('abeceda');
vypíše "ABECEDA"

UPPER převádí písmena zadaného řetězce na velká, **LOWER** na malá

9.3.18 Manipulace s datumem a časem

SELECT NOW();

Příkaz vrátí aktuální datum a čas ve tvaru RRRR-MM-DD hh:mm:ss.

Modifikace

SELECT NOW()+0 vrátí tvar RRRRMMDDhhmmss

SELECT CURRENT_DATE() - aktuální datum (RRRR-MM-DD)

SELECT CURRENT_TIME() - aktuální čas (hh:mm:ss)

SELECT DATE_FORMAT(vstup,vystup);

Příklad:: SELECT DATE_FORMAT(NOW(), "%w.%e.%y");

%Y - rok RRRR (př. 2003, 1999 ...)

%y - rok RR (př. 03, 99 ...)

%m - měsíc MM (př. 01, 06, 12 ...)

%c - měsíc M nebo MM (př. 1, 6, 12 ...)

%M - název měsíce (př. January ...)

%b - název měsíce zkráceně (př. Jan, Feb ...)

%u - číslo týdne v roce - %D - den řadovou číslovkou (př. 1st, 2nd ...)

%d - den v měsíci DD (př. 01, 02, 31 ...)

%e - den v měsíci D nebo DD (př. 1, 2, 31 ...)

%w - číslo dne v týdnu D (př. 0, 6 ...)

%W - název dne v týdnu (př. Sunday ...)

%a - název dne v týdnu zkráceně (př. Sun, Mon ...)

%j - číslo dne v roce DDD (př. 000, 006, 366 ...)

%H - hodina HH (př. 00, 06, 23 ...)

%k - hodina H nebo HH (př. 0, 6, 23 ...)

%h - hodina HH jen do 12 (př. 01, 06, 12 ...)

%l - hodina H nebo HH jen do 12 (př. 1, 6, 12 ...)

%i - minuty MM (př. 01, 06, 59 ...)

%s - sekundy SS (př. 01, 06, 59 ...)

%P - délka cyklu - půldenní nebo celodenní (př. AM, PM)

%% - %

SELECT QUARTER(datum);

Vrací číslovku čtvrtletí dle zadaného data (RRRR-MM-DD).

9.3.19 Výpis záznamů do souboru

SELECT * INTO OUTFILE 'navez_vystupniho_souboru' FIELDS TERMINATED BY ';' FROM navez_tabulky; - příkaz zapíše data z tabulky "navez_tabulky" do souboru a jednotlivé položky oddělí středníkem

Příklad: `SELECT * INTO OUTFILE 'prectene.txt' FIELDS TERMINATED BY ',' FROM knihovna WHERE poznamka='precteno';`
Příklad zapíše do souboru informace o přečtených knihách a oddělí je čárkou.

9.3.20 Mazání záznamů

DELETE FROM *nazev_tabulky* **WHERE** *podminka*;

Příklad: `DELETE FROM knihovna WHERE kniha='Horníkův den';`
Příkaz vymaže knihu "Horníkův den" z tabulky, tedy celý řádek. Zápis bez podmínky: `DELETE FROM` *název_tabulky*; smaže celou tabulku.

10 Nastavení relace mezi tabulkami MySQL

Mysam – nejde nastavit relace

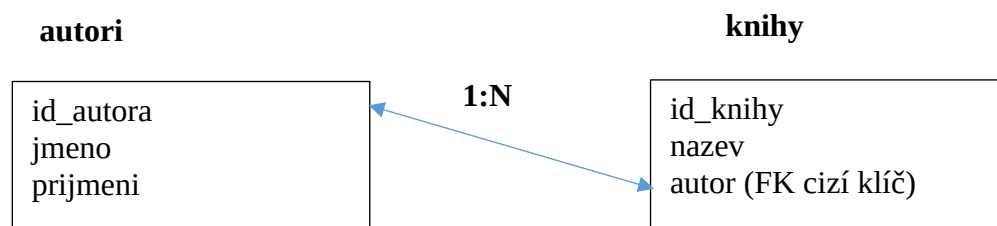
Innodb – lze nastavit relaci, tedy vztad primárního a cizího klíče mezi tabulkami.

10.1 Příklad

Tabulky

knihyfk ck, *nazev*, *autor*

autorifk ca, *jmeno*, *prijmeni*



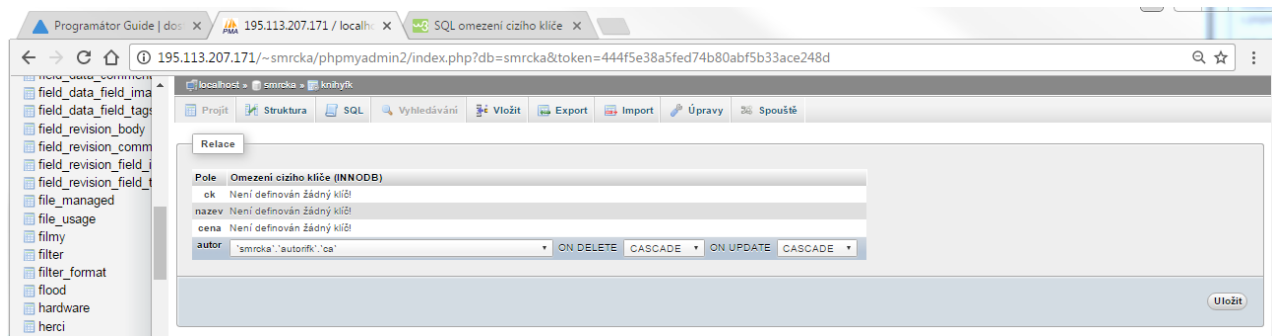
Sloupec autor je již vytvořený.

Pak dáme SQL dotaz

```
ALTER TABLE knihyfk
ADD FOREIGN KEY (autor)
REFERENCES autorifk(ca)
```

Sloupec autor je již vytvořený.

Dále je možné nastavit update a delete v kaskádě.



Místo SQL jde vytvořit cizí klíč v grafice

